DAILY BRIEF · DRAWING NO. 2026-1015

每日晨间简报

DATE
星期四 · 2026年10月15日

SHEET
项目 · 练习 · 重写 · 记录

SCALE
一步一步

今天

其实地上本没有路,走的人多了,也便成了路。

—— 鲁迅《故乡》(1921 年,收入小说集《呐喊》。小说的最后一句。)

TODAY'S SONG · 今日一首歌

红豆 — 王菲

1998 年的慢歌,林夕作词,编曲简单,只有一点吉他和弦乐,声音很轻。适合晚上做完练习、填记录的时候听。

今天做 chat-cli 第 5 天:命令行参数与中断,练习 CodeCrafters ih0 Extract URL path。

01 今日三条新闻

N-1

Character.AI 申请破产,40 位州总检察长要求保护用户聊天数据

TechCrunch 10月13日报道:聊天机器人平台 Character.AI 申请第 11 章破产保护,资产和负债各在 5,000 万到 1 亿美元之间;已同意以 2,000 万美元把部分资产卖给迪士尼,出售预计 11 月底或 12 月初完成。CEO 说,如果找不到收购整个平台的买家,平台将会关闭。据 AIHOT 整理的 The Verge 10月14日报道,纽约总检察长 Letitia James 联合 40 位两党总检察长在破产案中提交通知,要求任何资产出售都要落实用户数据保护。(AIHOT)

为什么值得看 → 你和聊天机器人说过的话存在公司的服务器上,公司倒了,这些记录就成了可以卖的“资产”。你的 chat-cli 历史只放在自己程序的内存里,是另一种做法。

N-2

LangChain 讲上下文工程:长任务里怎么“记住”又不塞满窗口

LangChain 10月14日发布博客,介绍开源智能体框架 Deep Agents 的五种做法:超过约 2 万词元的工具结果写进文件,模型只看一段预览和文件路径;对话接近上下文窗口的 85% 时,把较早的消息压缩成摘要,最近的消息原样保留;另外还有按需加载的 skills、在单独窗口里干活的子智能体,以及存在 markdown 文件里的长期记忆。(AIHOT)

为什么值得看 → 接着昨天的“上下文窗口”往下讲,今天英语精读就写它;你的 chat-cli 每轮重发全部历史,迟早要面对“该留什么”。

N-3

美国 9 月通胀高于预期,纳指跌 1.88%

TheStreet 10月14日收盘报道:9 月 CPI 年率 3.7%(预期 3.6%),核心 CPI 月率 0.3%(预期 0.2%)、年率 2.6%(预期 2.5%)。标普 500 跌 1.11% 收于 7,718.94,纳指跌 1.88% 收于 26,692.87,道指跌 0.09% 收于 51,621.48。报道说跌幅靠前的股票里有不少数据中心和 AI 相关公司;巴克莱下调戴尔、HPE 和 NetApp 评级,理由是估值偏高、企业 AI 带来的增长有限。

为什么值得看 → 通胀高,利率就可能更久维持在高位,最花钱的 AI 和算力公司先受影响,最后也会反映到模型和 API 的价格上。

所属方向:N-1 科技与 AI · N-2 编程与开发 · N-3 财经与市场

GITHUB 今日项目 · 你点名的

morluto / rea

你写的是“rea”,这是按名字在 GitHub 搜索接口里找到的、名字正好是 rea 的仓库;如果你说的是别的项目,在备注里写全名。

REA(Reverse Engineer Anything):让 AI 智能体去“拆”一个你没有源码的程序,弄清某个功能是怎么做的、给出证据,再在你自己的项目里重新实现一个。

TypeScript | 约 367 星(仓库接口) | 创建于 2026年4月14日 | MIT 许可证 | 早期项目,最近推送 2026年8月14日

§ 有什么用

- 三步:反编译(恢复能读的代码、字符串和名字)→ 理解(顺着代码讲清一个功能)→ 重建(用你自己的技术栈写一个)。README 明说不会还原原始源码,也不会自动复制整个程序。
- 同一套能力既有命令行,也有 MCP 服务器,可以接到 Claude Code、Codex、Cursor、Gemini CLI 等工具上用;分析在本机做,没有云端服务。

§ 为什么火

- 说实话不算火:四个月约 367 星、30 个 fork、41 个未关闭的 issue。另外,GitHub 搜索接口给它显示 238,573 星,和仓库接口的 367 对不上,这里以仓库接口为准,搜索里的数字不可信。
- 它的看点在方向:把 Hopper、Ghidra 这类逆向工具变成智能体能调用的工具,话题标签里有 mcp、agent-skills、decompiler。

§ 怎么用

```
npx -y rea-agents@latest doctor      # 检查环境
npx -y rea-agents@latest analyze /Applications/Notes.app
```

§ 要注意的

需要 Node.js 22.19+,原生分析还要另装 Hopper(单独授权的软件)或 Ghidra;README 说它不是安全沙箱,以当前用户的权限运行。只拿自己有权分析的软件试。以上来自 README 和 GitHub 统计,未实际安装。

§ 和你的关系

它最值得你看的是“MCP 服务器”那部分:你的 chat-cli 指南最后提到用 rmcp 写 MCP 服务器,就是这种东西的 Rust 版。

02 今日任务

标“必做”的是项目(chat-cli 第 5 天)、今日练习(今天是 CodeCrafters)、隔天重写和填写记录;其余选做,做到哪算哪。

打卡 · 复利

从 2026年10月10日 起算 · 截至昨天

×1.318 总累计

本月(10月):**×1.318**

本月 1.1 有 **4** 天,0.9 有 **1** 天

本月休息日:**0 / 4**

往月: —

已持续 **5** 天(10月10日起)

连续 1.1:第 **4** 天 · 状态:**未断**

2026年10月

一	二	三	四	五	六	日
			1	2	3	4
5	6	7	8	9	10 0.9	11 1.1
12 1.1	13 1.1	14 1.1	15 ?	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

实心深蓝 = 1.1 · 虚线框 = 0.9 · 点线框 = 今天 · 斜线 = 起算之前 · 细边空白 = 以后 · 右上角“休” = 休息日

☐ 今天:项目和今天的 CodeCrafters 都有实在的进展(完成,或部分完成并写了做到第几步),就是 1.1——总累计变成 **1.449**;差一样就是 0.9——变成 **1.186**。

回你的备注

14 日两样必做都完成了:chat-cli 第 4 天做完,1617 独立通过,还重写了 1092、骑了 44 分钟、读完第 7 章。昨天是 1.1,连续第 4 天。

项目连续两天选了“问了 AI”,第 4 天花了约两个半小时。那份 AI 总结里最该留下的是“要能随时说出每个变量的类型”。给你一个办法:不确定时写一行 `let _: () = item;`,编译器报错里的 found 后面就是它的真实类型,看完删掉。今天项目卡开头多加了两条方向提示;再去问 AI 时,先写一句“只给方向,先别给代码”。

重写 1092 时忘了 for_each 和 iter / iter_mut / into_iter 的关系——今天 Rust 第 5 讲就讲迭代器,把这三个一次讲清;ih0 切请求行用的 `split_whitespace().nth(1)` 也是迭代器。

《1984》那句出自温斯顿的日记。我的理解不只是“他们没有自由”,而是:最根本的自由,是能承认眼前的事实——二加二就是四,不管谁说不是。连这一点都不敢说、不能坚持,别的自由都无从谈起;守住了它,其他的才跟得上。党想控制的不只是人做什么,还有什么算“真的”。你往后读,这层意思会越来越清楚。rea 放在今天的 GitHub 卡片里了。

手写 ih0 时,先把 `curl` 发出的请求行和两种响应逐字写在纸上,`\r\n` 也写出来,再动手改代码。

从这里开始

chat-cli 第 5 天 第 1 条:用 clap 定义 `Args` 结构体,包含 `--model`(默认 `deepseek-flash`)和 `--system`(可选)。

A-01 ☐ chat-cli 项目 · 第 5 天:命令行参数与中断

必做

先看这里 · 两条方向提示

1. 今天其实是两件互不相干的事。先只做命令行参数(第 1–3 条),`cargo run -- --help` 能看到帮助、`--model` 真的换了模型,就 commit 一次,再碰 Ctrl+C。

2. 第 4 条不用重写循环,是“搬家”:原来 `while let Some(item) = stream.next().await { … }` 里的 `stream.next()` 原样放进 `select!` 的第一个分支;拿到 `Some(item)` 时执行原来大括号里的那些代码,拿到 `None` 时 `break`。你第 4 天用的 `break 'outer` 在分支里照样能用。

目标:程序能通过参数换模型和 system prompt;生成到一半按 Ctrl+C 只打断当前回复,不退出程序。

加依赖:

```
cargo add clap --features derive
```

§ 命令行参数

第1条 ☐ 用 clap 定义 `Args` 结构体,包含 `--model`(默认 `deepseek-flash`)和 `--system`(可选)

第2条 ☐ 用解析出的值替换代码里写死的模型名和 system prompt

第3条 ☐ 运行 `cargo run -- --help`,看 clap 自动生成的帮助信息

```
use clap::Parser;

#[derive(Parser)]
struct Args {
    #[arg(long, default_value = "deepseek-flash")]
    model: String,

    #[arg(long)]
    system: Option<String>,
}
```

传参数时注意中间的 `--`:`cargo run -- --model deepseek-v4-pro`。

§ Ctrl+C 中断

第4条 ☐ 把第 4 天的 `while let` 循环改成 `loop` 加 `tokio::select!`

第5条 ☐ 一个分支等数据流的下一块,另一个分支等 `tokio::signal::ctrl_c()`

第6条 ☐ 被打断时打印提示,跳出循环,回到输入提示符

第7条 ☐ 决定被打断的半截回复怎么处理:留在历史里,或者连同那条用户消息一起丢掉

骨架:

```
loop {
    tokio::select! {
        chunk = stream.next() => {
            // Some(...) 就处理这一块,None 就 break
        }
        _ = tokio::signal::ctrl_c() => {
            println!("\n[已中断]");
            break;
        }
    }
}
```

`select!` 的意思是:同时等几件事,哪件先发生就执行哪个分支。跳出循环后数据流被丢弃,连接随之关闭,服务器就不再继续生成了。

§ 常见坑

◇ 用了 `tokio::signal::ctrl_c()` 之后,Ctrl+C 不再能直接杀掉程序。在输入提示符下按 Ctrl+C 会没反应,退出请用 `/exit`。

◇ 如果选择保留半截回复,要确保它确实被追加进历史,否则下一轮会出现两条连续的 user 消息。

可选:请求体加 `"stream_options": {"include_usage": true}`,最后一个数据块会带上 `usage`,每轮结束后打印 token 用量。

> **完成标准** 用 `--model` 换模型能正常对话;让模型写一篇长文,中途按 Ctrl+C,输出停止并回到提示符,还能继续聊。

照你的 7 天指南 第 5 天写。做完后在记录页把项目标为完成,简报才会进到第 6 天;没做完就选部分完成并写做到第几条(不写不算进展),卡住时把报错也填进项目那一栏。

B-02 ☐ CodeCrafters HTTP 服务器 · ih0 Extract URL path

必做

关卡内容来自 GitHub 上公开的关卡说明(codecrafters-io/build-your-own-http-server),不是登录后的页面。

§ 思路

这一关第一次要“读”客户端发来的东西。请求的第一行叫请求行,形如 `GET /abcdefg HTTP/1.1`,中间那一段就是路径:路径是 `/` 就回 200,其他一律回 404。难点只有一个:怎么从连接里读出这一行,再把中间那段切出来。头部这一关可以不管。

1 **先看请求长什么样。**

在 `Ok(mut stream)` 分支里,先读出请求的第一行,原样打印(用 `{:?}` 打印,能看见 `\r\n`)。响应先照旧回 200。

> **怎么确认:**运行服务器,另开终端 `curl -v http://localhost:4221/abcdefg`,服务器那边打印 `"GET /abcdefg HTTP/1.1\r\n"`。

2 **切出路径。**

把这一行按空白切开,取第 2 段,打印出来。

> **怎么确认:**请求 `/abcdefg` 时打印 `/abcdefg`;请求 `http://localhost:4221/` 时打印 `/`,前后没有多余的空格或 `\r`。

3 **按路径选响应。**

路径等于 `"/"` 写回 200 那一行,否则写回 404 那一行。

> **怎么确认:**两次 curl 分别看到下面框里的两种状态行。

4 **别让怪请求把服务器弄崩。**

第 2 段可能不存在(比如空请求),这时当作 404,不要 `unwrap()`。

> **怎么确认:**想一想:切出来的类型是什么?`None` 时走哪条路?代码里没有对它 `unwrap()`。

5 **删掉调试打印,提交。**

去掉第 1、2 步的 println,`git commit` 后 `git push origin master`(或 `codecrafters submit`,看你平时怎么交)。

> **怎么确认:**网页上 ih0 显示通过。

```
$ curl -v http://localhost:4221/abcdefg
> GET /abcdefg HTTP/1.1
< HTTP/1.1 404 Not Found

$ curl -v http://localhost:4221/
> GET / HTTP/1.1
< HTTP/1.1 200 OK

服务器要写回的两种字节:
HTTP/1.1 200 OK\r\n\r\n
HTTP/1.1 404 Not Found\r\n\r\n
```

§ 相关知识

◇ **请求的结构:**请求行 `\r\n`,然后若干行头部(每行 `\r\n` 结尾),然后一个空行 `\r\n`,最后是可选的正文。这一关只读第一行。

◇ **读一行:**`BufReader::new(&stream)` 包一层,再 `read_line(&mut line)`;需要 `use std::io::{BufRead, BufReader};`。读到的行末尾带 `\r\n`。传 `&stream`(借用)是因为后面还要用 `stream` 写回响应——读完以后不再用 reader,再 `write_all`,编译器就不会抱怨。

◇ **切路径:**`split_whitespace()` 按空白切开,顺带把 `\r\n` 丢掉;`.nth(1)` 取第 2 段,返回 `Option<&str>`。兜底可以用 `unwrap_or("")`,空字符串不等于 `"/"`,自然走 404。

◇ **404 状态行:**`HTTP/1.1 404 Not Found`,“Not Found”中间有空格,大小写照写;结尾同样是两个 `\r\n`。

手写区

把 `curl /abcdefg` 时客户端发来的请求行、和服务器要写回的两种响应逐字写出来,`\r\n` 也写上;再用中文写出要改的几步。

明天:CSES 1618 Trailing Zeros。

B-03 ☐ 隔天重写 · CodeCrafters ia4 Respond with 200 必做

今天要重写的:10月13日做完的 HTTP 服务器 ia4 Respond with 200。

做法:合上原来的代码,新建文件从空白开始;先用中文注释写出每一步,再一行行翻成 Rust;运行后用 `curl -v http://localhost:4221` 看到 `HTTP/1.1 200 OK`;写不出来的地方记进记录页“重写时哪里又写不出来了”。重写完再在它上面接着做 ih0,正好顺下来。

§ 对照清单

☐ bind 到 127.0.0.1:4221,`incoming()` 循环,`match` 的 Ok / Err 两个分支

☐ `Ok(mut stream)` 为什么要 mut,以及 `use std::io::Write`

☐ 用 `write_all` 写出状态行和结尾的空行(两个 `\r\n`)

重写完在记录页勾上 CodeCrafters 那一项。

C-04 ☐ **Rust 知识点 · 第 5 讲:迭代器** 选做

一句话:迭代器就是“一个接一个往外给元素”的东西。同一个 Vec 可以借着给你看(`iter`)、借着让你改(`iter_mut`),或者整个交出去(`into_iter`)——这正是你重写 1092 时没想起来的那三个。

§ 第一部分 · 书里讲的

◇ **三种来源。** `v.iter()` 给出 `&T`(只读借用);`v.iter_mut()` 给出 `&mut T`(可改借用,v 要 `let mut`,改的时候写 `*x`);`v.into_iter()` 给出 `T` 本身,v 被用掉,之后不能再用。

◇ **for 循环背后就是迭代器。** `for x in &v` 等于 `v.iter()`;`for x in &mut v` 等于 `v.iter_mut()`;`for x in v` 等于 `v.into_iter()`。

◇ **适配器是“懒”的。** `map`、`filter` 只描述要做什么,不会自己跑;必须接一个“消费者”——`collect`、`sum`、`count`、`for_each`、`nth`——才真正一个个取元素。

◇ `for_each` 和 for 循环一样逐个处理,只是写成“方法 + 闭包”:`v.iter().for_each(|x| println!("{x}"))`,`|x|` 是闭包的参数。

◇ `collect` **要知道收集成什么。** 写成 `let lens: Vec<usize> = …collect();`,或者 `collect::<Vec<_>>()`。

例 1(能编译):三种用法依次来。输出 `1 2 3 [10, 20, 30] 60`,下一行 `10 20 30`。

```
fn main() {
    let mut v = vec![1, 2, 3];
    for x in v.iter() {
        print!("{x} ");
    }
    for x in v.iter_mut() {
        *x *= 10;
    }
    let total: i32 = v.iter().sum();
    println!("{:?} {total}", v);
    for x in v.into_iter() {
        print!("{x} ");
    }
    println!();
}
```

例 2(这段编不过):`into_iter()` 把 v 用掉了,后面又用 v。报 `E0382: borrow of moved value: `v``。改法:只想看就写 `v.iter()` 或 `for s in &v`。

```
fn main() {
    let v = vec![String::from("a"), String::from("b")];
    for s in v.into_iter() {
        println!("{s}");
    }
    println!("{}", v.len());
}
```

例 3(能编译):`for_each`、`map` + `collect`、`nth`。最后两行输出 `[3, 4, 8]` 和 `Some("/abc")`。

```
fn main() {
    let words = vec!["GET", "/abc", "HTTP/1.1"];
    words.iter().for_each(|w| println!("{w}"));
    let lens: Vec<usize> =
        words.iter().map(|w| w.len()).collect();
    println!("{:?}", lens);
    let path = words.iter().nth(1);
    println!("{:?}", path);
}
```

例 4(这段编不过):v 没写 `mut` 却要 `iter_mut()`。报 `E0596: cannot borrow `v` as mutable, as it is not declared as mutable`。

```
fn main() {
    let v = vec![1, 2, 3];
    for x in v.iter_mut() {
        *x += 1;
    }
    println!("{:?}", v);
}
```

§ 第二部分 · 书里没细讲,但你应该知道的

◇ **为什么要分三种。**它们正好对应所有权的三种状态:借着读、借着改、拿走——就是第 1 讲和第 4 讲的规则。Python、JavaScript 的 for 只有一种,因为它们没有“谁拥有这个值”的概念;Rust 把它写进了方法名。

◇ **不会更慢。**迭代器链编译后通常和手写的 for 循环一样快,`map`、`sum` 不会先复制出一个新数组。所以不必为了“性能”刻意避开它们。

◇ **`iter()` 给的是引用。**对 `Vec<String>` 调 `iter()` 拿到的是 `&String`;想得到一份自己的 String,要么 `.cloned()`(复制),要么 `into_iter()`(拿走)。你 13 日写的 `messages.iter().cloned().collect()` 就是这样得到 `Vec<Message>` 的。

◇ **`for_each` 里不能 `break`。**闭包里的 `return` 只结束这一次调用,不会跳出整个循环。需要中途停,就用普通 for 循环,或者 `find`、`any`、`position` 这类会自己停下的方法。

◇ **两种场景。**刷题时最常见的一条链是 `s.split_whitespace().map(|t| t.parse::<u64>().unwrap())`,一次读完所有数字;写服务器时 `line.split_whitespace().nth(1)` 取路径,以后读头部用 `lines()` 逐行处理。

§ 常见错误

◇ `E0382`:`into_iter()` 或 `for x in v` 之后又用 v → 改成 `&v` 或 `v.iter()`。

◇ 警告 `unused `Map` that must be used`:只写了 `map`,没有消费者,什么也没发生 → 后面接 `collect` 或 `for_each`。

§ 小练习

写一个函数 `fn path_of(line: &str) -> Option<&str>`,函数体只用一条迭代器链。`path_of("GET /echo/abc HTTP/1.1\r\n")` 要得到 `Some("/echo/abc")`,`path_of("")` 要得到 `None`。

和今天任务的联系:ih0 的第 2 步就是这道小练习:`split_whitespace()` 是迭代器,`nth(1)` 是消费者,结果是 `Option<&str>`。chat-cli 第 5 天里 `stream.next()` 也是“取下一个”,只是要 `.await`,这个叫异步流,原理和迭代器一样。重写 ia4 用不到这一讲。

练习答案:`line.split_whitespace().nth(1)`。 · 对应书里:13.2 使用迭代器处理元素序列。

§ Essay

Remembering Without Keeping Everything

¶1 Yesterday you read that a model's context window has a limit. But there is a second problem that comes before the limit. On October 14, LangChain wrote that when an agent works for a long time, its context fills up with old messages and large tool results, and its accuracy drops. They call this **context rot**.

¶2 The answer is called **context engineering**: deciding, for every call, what goes into the window and what stays outside. LangChain's open-source agent harness, Deep Agents, uses five techniques. The first is **offloading**. When a tool result is larger than about 20,000 tokens, it is written to a file. The model only sees a short **preview** and the file path, and it can read the file later if it needs to.

¶3 The second is summarization. When the conversation reaches 85 percent of the window, older messages are **summarized**, while the most recent messages stay **verbatim**. The full history is still saved to a file, so nothing is really lost. The third and fourth are skills, which load their full instructions only when they are needed, and **subagents**, which do noisy work in their own window and return only the final result.

¶4 The fifth is long-term memory: a markdown file that is loaded at the start of every run. Because the file lives in storage you control, the memory can **persist** even if you switch to another model.

¶5 Your chat-cli has the simplest kind of memory: a Vec of messages that grows every turn. One day you may keep the system message and the last ten messages, and replace the rest with a short summary. The idea is the same as Deep Agents. Memory is not the window. Memory is what you choose to put back into it.

§ 8 个关键词

context rot 上下文腐烂(上下文越堆越乱,效果变差)

context engineering 上下文工程

offload 转移出去、卸到别处

preview 预览

summarize 总结、压缩成摘要

verbatim 逐字地、原样地

subagent 子智能体

persist 持续保存下来

新闻原文:Context Engineering with Deep Agents: How To Remember Without Keeping Everything In Context(LangChain 博客);AIHOT 条目。

用中文写 3 句话总结:

§ 中文对照(先读完英文再看)• 记住,但不全留着

¶1 昨天你读到,模型的上下文窗口有上限。但在碰到上限之前,还有第二个问题。10 月 14 日,LangChain 写道:智能体工作时间一长,上下文里就塞满了旧消息和很大的工具结果,准确率会下降。他们把这叫作**上下文腐烂**。

¶2 对策叫**上下文工程**:每一次调用都要决定,什么放进窗口、什么留在外面。LangChain 的开源智能体框架 Deep Agents 用了五种做法。第一种是**转移出去**:工具结果超过约 2 万词元时就写进文件,模型只看到一段简短的**预览**和文件路径,需要时再去

读那个文件。

¶3 第二种是总结:对话达到窗口的 85% 时,较早的消息会被**压缩成摘要**,最近的消息则**原样**保留。完整历史仍然存在一个文件里,所以其实什么都没丢。第三和第四种是 skills——只在需要时才加载完整说明——和**子智能体**——在自己的窗口里做杂乱的活,只把最终结果交回来。

¶4 第五种是长期记忆:一个在每次运行开始时加载的 markdown 文件。因为这个文件存在你自己掌控的地方,即使换成别的模型,记忆也能**保存下来**。

¶5 你的 chat-cli 有最简单的一种记忆:一个每轮都在变长的消息 Vec。将来你也许会只保留 system 消息和最近十条,把其余的换成一段简短的摘要。这和 Deep Agents 是同一个思路。记忆不是窗口;记忆是你选择重新放回窗口里的东西。

先读完英文,再看关键词,最后对照中文。

今天读:Chapter 1 Introduction · 1.3 Working with numbers · 书页 6–8(PDF 第 16–18 页)

§ Before you read

Q1 Which integer type can you use when int is not big enough?

Q2 Why do some problems ask for the answer "modulo $10^9 + 7$"?

1.3 Working with numbers

Integers

¶1 The most used integer type in competitive programming is int, which is a 32-bit type with a value range of $-2^{31} \ldots 2^{31} - 1$ or about $-2 \cdot 10^9 \ldots 2 \cdot 10^9$. If the type int is not enough, the 64-bit type long long can be used. It has a value range of $-2^{63} \ldots 2^{63} - 1$ or about $-9 \cdot 10^{18} \ldots 9 \cdot 10^{18}$.

¶2 The following code defines a long long variable:

```
long long x = 123456789123456789LL;
```

¶3 The suffix LL means that the type of the number is long long.

¶4 A common mistake when using the type long long is that the type int is still used somewhere in the code. For example, the following code contains a subtle error:

```
int a = 123456789;
long long b = a*a;
cout << b << "\n"; // -1757895751
```

¶5 Even though the variable b is of type long long, both numbers in the expression a*a are of type int and the result is also of type int. Because of this, the variable b will contain a wrong result. The problem can be solved by changing the type of a to long long or by changing the expression to (long long)a*a.

¶6 Usually contest problems are set so that the type long long is enough. Still, it is good to know that the g++ compiler also provides a 128-bit type __int128_t with a value range of $-2^{127} \ldots 2^{127} - 1$ or about $-10^{38} \ldots 10^{38}$. However, this type is not available in all contest systems.

Modular arithmetic

¶7 We denote by x mod m the remainder when x is divided by m. For example, 17 mod 5 = 2, because $17 = 3 \cdot 5 + 2$.

¶8 Sometimes, the answer to a problem is a very large number but it is enough to output it ”modulo m”, i.e., the remainder when the answer is divided by m (for example, ”modulo $10^9 + 7$”). The idea is that even if the actual answer is very large, it suffices to use the types int and long long.

¶9 An important property of the remainder is that in addition, subtraction and multiplication, the remainder can be taken before the operation:

$$(a + b) \bmod m = (a \bmod m + b \bmod m) \bmod m$$
$$(a - b) \bmod m = (a \bmod m - b \bmod m) \bmod m$$
$$(a \cdot b) \bmod m = (a \bmod m \cdot b \bmod m) \bmod m$$

¶10 Thus, we can take the remainder after every operation and the numbers will never become too large.

¶11 For example, the following code calculates n!, the factorial of n, modulo m:

```
long long x = 1;
for (int i = 2; i <= n; i++) {
    x = (x*i)%m;
}
cout << x%m << "\n";
```

¶12 Usually we want the remainder to always be between $0 \ldots m - 1$. However, in C++ and other languages, the remainder of a negative number is either zero or negative. An easy way to make sure there are no negative remainders is to first calculate the remainder as usual and then add m if the result is negative:

```
x = x%m;
if (x < 0) x += m;
```

¶13 However, this is only needed when there are subtractions in the code and the remainder may become negative.

Floating point numbers

¶14 The usual floating point types in competitive programming are the 64-bit double and, as an extension in the g++ compiler, the 80-bit long double. In most cases, double is enough, but long double is more accurate.

¶15 The required precision of the answer is usually given in the problem statement. An easy way to output the answer is to use the printf function and give the number of decimal places in the formatting string. For example, the following code prints the value of x with 9 decimal places:

```
printf("%.9f\n", x);
```

¶16 A difficulty when using floating point numbers is that some numbers cannot be represented accurately as floating point numbers, and there will be rounding errors. For example, the result of the following code is surprising:

```
double x = 0.3*3+0.1;
printf("%.20f\n", x); // 0.99999999999999988898
```

¶17 Due to a rounding error, the value of x is a bit smaller than 1, while the correct value would be 1.

¶18 It is risky to compare floating point numbers with the == operator, because it is possible that the values should be equal but they are not because of precision errors. A better way to compare floating point numbers is to assume that two numbers are equal if the difference between them is less than ε, where ε is a small number.

¶19 In practice, the numbers can be compared as follows ($\varepsilon = 10^{-9}$):

```
if (abs(a-b) < 1e-9) {
    // a and b are equal
}
```

¶20 Note that while floating point numbers are inaccurate, integers up to a certain limit can still be represented accurately. For example, using double, it is possible to accurately represent all integers whose absolute value is at most 2^{53}.

Antti Laaksonen, *Competitive Programmer's Handbook*, 第 1 章 1.3 节,CC BY-NC-SA 4.0。

§ 8 个关键词

value range 取值范围
suffix 后缀
subtle 不易察觉的
denote 表示、记作
remainder 余数
modulo 取模、对……取余
rounding error 舍入误差
precision 精度

§ After you read

Q1 Why does b get a wrong value in the code with int a = 123456789?

Q2 Why can we take the remainder after every operation?

Q3 Why is it risky to compare floating point numbers with ==?

Write one sentence in English to sum up this section:

§ 中文对照(先读完英文再看)

¶1 算法竞赛里最常用的整数类型是 int,它是 32 位类型,取值范围是 $-2^{31} \cdots 2^{31}-1$,大约 $-2 \cdot 10^9 \cdots 2 \cdot 10^9$。如果 int 不够用,可以用 64 位的 long long,它的取值范围是 $-2^{63} \cdots 2^{63}-1$,大约 $-9 \cdot 10^{18} \cdots 9 \cdot 10^{18}$。

¶2 下面的代码定义了一个 long long 变量:〔代码见原文〕

¶3 后缀 LL 表示这个数的类型是 long long。

¶4 使用 long long 时常犯的一个错误是:代码里某处仍然用的是 int。例如,下面的代码里有一个不易察觉的错误:〔代码见原文〕

¶5 虽然变量 b 是 long long 类型,但表达式 a*a 里的两个数都是 int,结果也是 int。因此 b 里会是一个错误的结果。解决办法是把 a 的类型改成 long long,或者把表达式改成 (long long)a*a。

¶6 通常比赛题目会设计成 long long 就够用。不过也值得知道,g++ 编译器还提供 128 位的类型 __int128_t,取值范围是 $-2^{127} \cdots 2^{127}-1$,大约 $-10^{38} \cdots 10^{38}$。但并不是所有比赛系统都能用这个类型。

¶7 我们用 x mod m 表示 x 除以 m 的余数。例如 17 mod 5 = 2,因为 17 = 3 · 5 + 2。

¶8 有时,题目的答案是一个非常大的数,但只要输出它“模 m”的结果就行,也就是答案除以 m 的余数(例如“模 $10^9 + 7$”)。这样做的用意是:即使真正的答案非常大,用 int 和 long long 类型也就够了。

¶9 余数有一个重要性质:在加法、减法和乘法中,可以先取余再做运算:〔公式见原文〕

¶10 因此,我们可以在每次运算之后都取一次余数,数就永远不会变得太大。

¶11 例如,下面的代码计算 n 的阶乘 n! 模 m 的值:〔代码见原文〕

¶12 通常我们希望余数总是在 0 ⋯ m − 1 之间。但在 C++ 和其他一些语言里,负数的余数要么是 0,要么是负数。一个保证不出现负余数的简单办法是:先照常取余,如果结果是负数就再加上 m:〔代码见原文〕

¶13 不过,只有代码里有减法、余数可能变成负数时才需要这样做。

¶14 算法竞赛里常用的浮点类型是 64 位的 double,以及 g++ 编译器作为扩展提供的 80 位 long double。大多数情况下 double 就够了,但 long double 更精确。

¶15 答案需要的精度通常写在题目里。一个输出答案的简单办法是用 printf 函数,并在格式字符串里给出小数位数。例如,下面的代码把 x 的值打印到小数点后 9 位:〔代码见原文〕

¶16 使用浮点数的一个难处是:有些数无法用浮点数精确表示,会产生舍入误差。例如,下面这段代码的结果出人意料:〔代码见原文〕

¶17 由于舍入误差,x 的值比 1 略小一点,而正确的值应该是 1。

¶18 用 == 运算符比较浮点数是有风险的,因为两个值本应相等,却可能因为精度误差而不相等。比较浮点数更好的办法是:如果两个数的差小于 ε(一个很小的数),就认为它们相等。

¶19 实际中可以这样比较两个数($\varepsilon = 10^{-9}$):〔代码见原文〕

¶20 注意,虽然浮点数不精确,但在一定范围内的整数仍然可以被精确表示。例如,用 double 可以精确表示所有绝对值不超过 2^{53} 的整数。

译文据 CC BY-NC-SA 4.0 原文翻译,仅供个人学习。

§ 这一节的代码

书里的 C++ 代码讲了四件事:int 相乘会溢出、每乘一次就取余、负数取余要修正、浮点数不能直接用 == 比较。Rust 里对应的写法:先把两个数都转成 `i64` 再相乘(Rust 在调试模式下遇到 i32 溢出会直接 panic,而不是悄悄得到一个错数);取余和 C++ 一样用 `%`;想要永远不为负的余数,用 `rem_euclid`;浮点比较用 `(a - b).abs() < 1e-9`。下面这段依次输出 `15241578750190521`、`146326063`(20! 模 10^9+7)、`3`、`true`。

```rust
fn main() {
    let a: i32 = 123456789;
    let b = a as i64 * a as i64; // 先转成 i64 再乘
    println!("{b}");
    const M: i64 = 1_000_000_007;
    let mut x: i64 = 1;
    for i in 2..=20 {
        x = x * i % M;
    }
    println!("{x}");
    println!("{}", (-7i64).rem_euclid(5));
    let y: f64 = 0.3 * 3.0 + 0.1;
    println!("{}", (y - 1.0).abs() < 1e-9);
}
```

能用在哪类 CSES 题上:所有“答案对 10^9+7 取模”的计数题——你做过的 1617 Bit Strings 就是;还有像 1072 Two Knights 那样中间结果超过 $2 \cdot 10^9$、必须用 64 位整数的题。

路线:今天是 118 段里的第 3 段。上一段:1.2 Input and output,讲读输入、写输出和怎么让它们更快。下一段:1.4 Shortening code,讲 typedef 和宏这些缩短代码的写法。

Reference answers: ① Because a*a is computed as int, so it overflows before it is stored in b. ② Because in addition, subtraction and multiplication the remainder can be taken before the operation. ③ Because of rounding errors, two values that should be equal may be a little different. · Reference summary: Use long long when int is too small, take the remainder after every operation, and compare floating point numbers with a small ε instead of ==.

- [ ] **读书:《1984》第一部 第 8 章、第二部 第 1 章** —— 接着你读完的第 7 章,第一部到这里结束。第 8 章他一个人走进无产者住的街区,留意他想从一位老人嘴里问出什么,又在一家旧货店里买下了什么;第二部第 1 章是一个新的开始,先不多说。 选做

- [ ] **运动:游泳(和骑行换一天)** —— 你昨天已经骑了 44 分钟,今天换成游泳:先在水里慢走或慢游 100 米热身;自由泳或蛙泳 25 米 × 8–10 个,每个之间歇 30–60 秒;最后放松游 100 米,上岸拉伸肩和腿。 选做

Z-99 ☐ **填写今晚的记录** 必做

项目填在记录页最上面的项目一栏,今天的练习填 CodeCrafters 那一栏(关卡填 ih0);没做完就选部分完成并写做到第几步,不写不算进展,打卡就是 0.9;照实选“怎么完成的”,它只用来调提示多少;重写了就勾上 CodeCrafters 那一项;选做的做了就勾(精读和原著各有一个勾);想休息就在记录页“宣布休息日”一栏选日期,至少提前一天(本月还剩 4 天)。备注里想说什么都可以写,明天的简报开头会回你;明天从这里写的位置接着排,打卡系数也按这里算。